Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 15 (1)

Roll No.: 352

Name: Gohel Smit

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

In [1]:
```
class Students:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

stu1 = Students("smit",1,"O")

print("Name:", stu1.name)
print("Age:", stu1.age)
print("Grade:", stu1.grade)
```

```
Name: smit
Age: 1
Grade: O
```

02) Create a class named Bank_Account with Account_No, User_Name, Email,Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

In [5]:
```
class Bank_Account:
    def __init__(self):
        self.Account_No = ""
        self.User_Name = ""
        self.Email = ""
        self.Account_Type = ""
        self.Account_Balance = 0

    def get_account_details(self):
        self.Account_No = input("Enter Account Number: ")
        self.User_Name = input("Enter User Name: ")
        self.Email = input("Enter Email: ")
        self.Account_Type = input("Enter Account Type: ")
        self.Account_Balance = float(input("Enter Account Balance: "))

    def display_account_details(self):
        print("--- Account Details ---")
        print("Account No:", self.Account_No)
        print("User Name:", self.User_Name)
        print("Email:", self.Email)
        print("Account Type:", self.Account_Type)
        print("Balance:", self.Account_Balance)

if __name__ == "__main__":
    acc1 = Bank_Account()
    acc1.get_account_details()
    acc1.display_account_details()
```

```
--- Account Details ---
Account No: 101
User Name: Smit
Email: smit@gmail.com
Account Type: Savings
Balance: 7.0
```

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

In [9]:
```python
import math
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.radius

c1 = Circle(5)
print(f"Radius: {c1.radius}")
print(f"Area: {c1.area():.2f}")
print(f"Perimeter: {c1.perimeter():.2f}")
```

```
Radius: 5
Area: 78.54
Perimeter: 31.42
```

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and display employee information.

In [15]:
```python
class Employees:
    def __init__(self):
        self.items = {}

    def update_info(self, name, age, salary):
        self.items[name] = {"age": age, "salary": salary}

    def display_info(self):
        for name, details in self.items.items():
            print(f"Name: {name}, Age: {details['age']}, Salary: {details['salary']}")

e = Employees()
e.update_info("Smit", 1, 7000)
e.update_info("mit", 2, 10000)
e.display_info()
```

```
Name: Smit, Age: 1, Salary: 7000
Name: mit, Age: 2, Salary: 10000
```

05) Create a bank account class with methods to deposit, withdraw, and check balance.

In [16]:
```python
class Bank:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}")
        else:
            print("Invalid deposit amount")

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount")
        elif self.balance >= amount:
            self.balance -= amount
            print(f"Withdrawn: {amount}")
        else:
            print("Insufficient Balance")

    def check_balance(self):
        print(f"Current Balance: {self.balance}")

acc = Bank(5000)

acc.deposit(2000)
acc.withdraw(3000)
acc.withdraw(5000) # show insufficient
acc.check_balance()
```

```
Deposited: 2000
Withdrawn: 3000
Insufficient Balance
Current Balance: 4000
```

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

In [17]:
```python
class Inventory:
    def __init__(self):
```

```python
        self.items = {}

    def add(self, name, price, quantity):
        if price < 0 or quantity < 0:
            print("Invalid price or quantity")
            return

        self.items[name] = {"price":price, "quantity":quantity}
        print(f"{name} added successfully")

    def remove(self, name):
        if name in self.items:
            del self.items[name]
            print(f"{name} removed")
        else:
            print(f"{name} doesn't exist")

    def update(self, name, price, quantity):
        if name in self.items:
            if price < 0 or quantity < 0:
                print("Invalid price or quantity")
                return

            self.items[name] = {"price": price, "quantity": quantity}
            print(f"{name} updated")
        else:
            print(f"{name} doesn't exist")

    def display(self):
        print("--- Inventory ---")
        for name, details in self.items.items():
            print(f"Item: {name}, Price: {details['price']}, Quantity: {details['quantity']}")

inv = Inventory()

inv.add("Laptop", 50000, 10)
inv.add("Mouse", 500, 50)

inv.display()

inv.update("Mouse", 600, 40)
inv.remove("Laptop")

inv.display()
```

```
Laptop added successfully
Mouse added successfully
--- Inventory ---
Item: Laptop, Price: 50000, Quantity: 10
Item: Mouse, Price: 500, Quantity: 50
Mouse updated
Laptop removed
--- Inventory ---
Item: Mouse, Price: 600, Quantity: 40
```

07) Create a Class with instance attributes of your choice.

In [19]:

```python
class Car:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

    def display(self):
        print(f"Car brand:{self.brand}, model:{self.model}, price:{self.price}")

    def apply_discount(self, percent):
        self.price -= self.price * (percent / 100)

c1 = Car("Mercedes-Benz", "S-Class", "₹2 Crore")
c2 = Car("Tesla", "Cybertruck", "₹50-80 Lakh")

c1.display()
c2.display()
```

```
Car brand:Mercedes-Benz, model:S-Class, price:₹2 Crore
Car brand:Tesla, model:Cybertruck, price:₹50-80 Lakh
```

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

In [20]:
```
class Student_kit:
    principal_name = "Blackberry"

    def __init__(self, name):
        self.days = 0
        self.name = name

    def attendance(self, no_of_days):
        self.days = no_of_days

    def certificate(self):
        print("--- Certificate ---")
        print(f"Student Name: {self.name}")
        print(f"Days Present: {self.days}")
        print(f"Principal: {Student_kit.principal_name}")

s1 = Student_kit(input("Enter Student Name: "))
days = int(input("Enter number of days present: "))

s1.attendance(days)
s1.certificate()
```

```
--- Certificate ---
Student Name: smit
Days Present: 7
Principal: Blackberry
```

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

In [23]:
```
class Time:
    def __init__(self, hour, minute):
        self.hour = hour
        self.minute = minute

    def add(self, t2):
        total_minutes = self.minute + t2.minute
        total_hours = self.hour + t2.hour

        if total_minutes >= 60:
            total_hours += total_minutes // 60
            total_minutes = total_minutes % 60

        return Time(total_hours, total_minutes) # Create new object

    def display(self):
        print(f"{self.hour} hour {self.minute} minutes")

t1 = Time(2, 50)
t2 = Time(1, 30)

t3 = t1.add(t2)

print("Time 1:")
t1.display()

print("Time 2:")
t2.display()

print("After Addition:")
t3.display()
```

```
Time 1:
2 hour 50 minutes
Time 2:
1 hour 30 minutes
After Addition:
4 hour 20 minutes
```